

Guia explicada de comandos: demo CP vs AP

Banco CP con MongoDB Replica Set y red social AP con Cassandra

Esta guia no solo lista comandos: tambien explica que hace cada linea y que puedes responder si el profesor pregunta.

Escenario	Sistema	Idea clave
Aplicacion bancaria CP	MongoDB Replica Set	Prioriza consistencia. Durante una particion, si no hay mayoria o PRIMARY, bloquea escrituras.
Red social AP	Cassandra	Prioriza disponibilidad. Con CONSISTENCY ONE acepta escrituras aunque un nodo este aislado.

1. Preparacion inicial

Estos comandos preparan la terminal y verifican que Docker esta funcionando.

```
cd "C:\Users\Usuario\OneDrive\Documentos\4to Semestre\base de datos distribuida\Actividad-viernes"
```

Que hace: Entra a la carpeta donde esta el archivo docker-compose.yml.

Que decir: Primero me ubico en la carpeta del proyecto, porque Docker Compose busca ahi la definicion de los servicios.

```
docker version
```

Que hace: Muestra la version del cliente y del servidor Docker.

Que decir: Con esto confirmo que Docker Desktop esta abierto y que la terminal puede comunicarse con Docker.

```
docker compose version
```

Que hace: Verifica que el subcomando Docker Compose esta disponible.

Que decir: Uso Compose porque mi infraestructura esta definida en un yml, no voy a crear contenedores uno por uno.

2. Levantar la maqueta desde cero

Esta seccion inicia todos los contenedores definidos en el yml.

```
docker compose down -v
```

Apaga y elimina los contenedores, la red y los volúmenes de datos creados por Compose.

Este comando es opcional. Lo uso si quiero empezar limpio y evitar datos o configuraciones anteriores.

docker compose up -d

Crea y levanta en segundo plano los servicios del yml: tres MongoDB y dos Cassandra.

Aquí levanto la infraestructura completa de la demo: MongoDB para CP y Cassandra para AP.

docker ps

Lista los contenedores que están corriendo.

Verifico que los nodos están activos antes de ejecutar operaciones sobre ellos.

Start-Sleep -Seconds 90

Hace una pausa de 90 segundos en PowerShell.

Cassandra tarda más en arrancar que MongoDB, por eso espero antes de consultarlo.

docker exec -it cassandra1 nodetool status

Ejecuta nodetool dentro del contenedor cassandra1 para ver el estado del cluster.

UN significa Up/Normal; si veo UN, Cassandra está listo para la demo.

3. Inicializar MongoDB Replica Set

Este paso es obligatorio en una computadora limpia. Evita el error no replset config has been received.

docker exec -it mongo1 mongosh --eval

```
"rs.initiate({_id:'rs0',members:[{_id:0,host:'mongo1:27017',priority:2},{_id:1,host:'mongo2:27017'},{_id:2,host:'mongo3:27017'}]}")"
```

Entra a mongo1 y crea el Replica Set llamado rs0 con tres miembros: mongo1, mongo2 y mongo3.

Este comando convierte los tres MongoDB independientes en un cluster con replica set.

MongoDB necesita esto para elegir un PRIMARY y replicar datos.

Start-Sleep -Seconds 10

Espera a que MongoDB termine la elección interna del PRIMARY.

Después de iniciar el Replica Set, doy unos segundos para que los nodos acuerden quién será el líder.

```
docker exec -it mongo1 mongosh --eval "rs.status().members.map(m =>
({name:m.name,state:m.stateStr}))"
```

Consulta el estado de cada miembro del Replica Set y muestra si es PRIMARY o SECONDARY.

Solo el PRIMARY acepta escrituras. Este comando me dice en que nodo debo insertar datos.

4. Escenario CP: aplicacion bancaria con MongoDB

El banco necesita consistencia: no debe aceptar una transferencia si no puede garantizar mayoria.

```
docker exec -it mongo1 mongosh --eval
"db.getSiblingDB('banco').cuentas.insertOne({cuenta:'A001',saldo:1000,operacion:'depo
sito inicial'},{writeConcern:{w:'majority',wtimeout:5000}})"
```

Inserta una operacion en la base banco, coleccion cuentas, exigiendo confirmacion de la mayoria.

El writeConcern majority representa el comportamiento bancario: la escritura solo se acepta si la mayoria del cluster la confirma.

```
docker exec -it mongo1 mongosh --eval
"db.getSiblingDB('banco').cuentas.find().toArray()"
```

Consulta los documentos guardados en la coleccion cuentas.

Con esto muestro que la operacion bancaria inicial fue registrada correctamente.

```
docker network disconnect actividad-viernes_default mongo2
```

Saca a mongo2 de la red Docker de la demo.

Estoy simulando una particion de red: mongo2 deja de comunicarse con los otros nodos.

```
docker network disconnect actividad-viernes_default mongo3
```

Saca a mongo3 de la red Docker de la demo.

Ahora mongo1 queda aislado de la mayoria, por eso MongoDB no puede garantizar consenso.

```
Start-Sleep -Seconds 15
```

Espera a que MongoDB detecte la perdida de comunicacion entre nodos.

Doy tiempo para que el Replica Set note la particion y cambie su estado interno.

```
docker exec -it mongo1 mongosh --eval
"db.getSiblingDB('banco').cuentas.insertOne({cuenta:'A001',saldo:500,operacion:'transf
erencia durante particion'},{writeConcern:{w:'majority',wtimeout:5000}})"
```

Intenta registrar una transferencia durante la particion, otra vez exigiendo mayoria.

Aqui deberia fallar o bloquearse. Eso demuestra CP: MongoDB prefiere no escribir antes que crear datos inconsistentes.

docker network connect actividad-viernes_default mongo2

Reconecta mongo2 a la red Docker.

Empiezo la recuperacion de la particion devolviendo los nodos a la red.

docker network connect actividad-viernes_default mongo3

Reconecta mongo3 a la red Docker.

Con mongo2 y mongo3 reconectados, el Replica Set puede volver a tener mayoría.

Start-Sleep -Seconds 20

Da tiempo a MongoDB para estabilizar el Replica Set tras la reconexion.

Espero para que MongoDB vuelva a elegir o confirmar un PRIMARY.

```
docker exec -it mongo1 mongosh --eval "rs.status().members.map(m =>
({name:m.name,state:m.stateStr}))"
```

Revisa otra vez que nodo quedo como PRIMARY.

Despues de una falla, el PRIMARY puede cambiar. Por eso verifico antes de escribir.

```
docker exec -it mongo1 mongosh --eval
"db.getSiblingDB('banco').cuentas.insertOne({cuenta:'A001',saldo:1500,operacion:'depo
sito despues de recuperacion'},{writeConcern:{w:'majority',wtimeout:5000}})"
```

Intenta insertar una operacion despues de recuperar la red. Debe ejecutarse en el nodo PRIMARY.

Si mongo1 es PRIMARY, este comando demuestra que el sistema se recupero y vuelve a aceptar escrituras consistentes.

```
docker exec -it mongo2 mongosh --eval
"db.getSiblingDB('banco').cuentas.insertOne({cuenta:'A001',saldo:1500,operacion:'depo
sito despues de recuperacion'},{writeConcern:{w:'majority',wtimeout:5000}})"
```

Mismo insert, pero apuntando a mongo2 por si mongo2 quedo como PRIMARY.

No se ejecutan todos; se usa solo el nodo que aparezca como PRIMARY. Si sale not primary, pruebo con el lider correcto.

```
docker exec -it mongo3 mongosh --eval
"db.getSiblingDB('banco').cuentas.insertOne({cuenta:'A001',saldo:1500,operacion:'depo
sito despues de recuperacion'},{writeConcern:{w:'majority',wtimeout:5000}})"
```

Mismo insert, pero apuntando a mongo3 por si mongo3 quedo como PRIMARY.

Esto refuerza que en MongoDB Replica Set las escrituras van al PRIMARY, no a cualquier nodo.

5. Preparar Cassandra para el escenario AP

Cassandra se usa como red social porque puede seguir disponible con consistencia flexible.

docker exec -it cassandra1 nodetool status

Muestra el estado de los nodos Cassandra.

Compruebo que Cassandra esta operativo antes de crear la tabla de posts.

docker exec -it cassandra1 cqlsh -e "CREATE KEYSPACE IF NOT EXISTS redsocial WITH replication = {'class':'SimpleStrategy','replication_factor':2};"

Crea el keyspace redsocial y configura que los datos se repliquen en dos nodos.

El keyspace representa la base de datos de la red social, y el replication factor 2 indica que los datos se copian en dos nodos.

docker exec -it cassandra1 cqlsh -e "CREATE TABLE IF NOT EXISTS redsocial.posts (id text PRIMARY KEY, usuario text, contenido text);"

Crea una tabla simple para publicaciones con id, usuario y contenido.

Esta tabla representa los posts de una red social.

docker exec -it cassandra1 cqlsh -e "CONSISTENCY ONE; INSERT INTO redsocial.posts (id, usuario, contenido) VALUES ('1','ana','post antes de particion');"

Inserta un post exigiendo respuesta de solo un nodo.

CONSISTENCY ONE prioriza disponibilidad: basta con que un nodo acepte la escritura.

docker exec -it cassandra1 cqlsh -e "CONSISTENCY ONE; SELECT * FROM redsocial.posts;"

Consulta los posts desde cassandra1 usando consistencia ONE.

Muestro que el primer post fue guardado correctamente.

6. Escenario AP: red social con Cassandra

La red social prefiere seguir aceptando publicaciones aunque un nodo quede separado temporalmente.

docker network disconnect actividad-viernes_default cassandra2

Aisla cassandra2 de la red Docker.

Simulo una particion de red: un nodo Cassandra queda fuera, pero cassandra1 sigue funcionando.

docker exec -it cassandra1 cqlsh -e "CONSISTENCY ONE; INSERT INTO redsocial.posts (id, usuario, contenido) VALUES ('2','ana','post durante particion');"

Inserta un post durante la particion aceptando confirmacion de un solo nodo.

Este comando demuestra AP: aunque haya particion, Cassandra acepta la publicacion porque todavia hay un nodo disponible.

docker exec -it cassandra1 cqlsh -e "CONSISTENCY ONE; SELECT * FROM redsocial.posts;"

Consulta los posts visibles desde cassandra1 durante la particion.

Muestro que la red social sigue respondiendo y ve el post nuevo.

docker network connect actividad-viernes_default cassandra2

Reconecta cassandra2 a la red Docker.

Termino la particion y vuelvo a unir el nodo al cluster.

Start-Sleep -Seconds 30

Espera a que Cassandra detecte de nuevo al nodo reconectado.

Doy tiempo para que el cluster establezca la comunicacion.

docker exec -it cassandra1 nodetool repair redsocial

Fuerza una reparacion del keyspace redsocial para sincronizar replicas.

Despues de recuperar la red, ejecuto repair para que los nodos se pongan al dia.

docker exec -it cassandra1 cqlsh -e "CONSISTENCY ONE; SELECT * FROM redsocial.posts;"

Consulta los posts desde cassandra1 despues de la recuperacion.

Verifico que cassandra1 mantiene los datos.

docker exec -it cassandra2 cqlsh -e "CONSISTENCY ONE; SELECT * FROM redsocial.posts;"

Consulta los posts desde cassandra2 despues de reconectarlo.

Verifico que cassandra2 tambien puede leer los datos tras la recuperacion.

7. Limpieza

Estos comandos sirven para repetir la demo sin datos acumulados.

docker exec -it mongo1 mongosh --eval "db.getSiblingDB('banco').cuentas.deleteMany({})"

Borra los documentos de la coleccion cuentas si mongo1 es PRIMARY.

Uso deleteMany para limpiar datos de la demo bancaria sin borrar el cluster.

docker exec -it mongo2 mongosh --eval

"db.getSiblingDB('banco').cuentas.deleteMany({})"

Borra los documentos desde mongo2 si mongo2 es PRIMARY.

Solo el PRIMARY puede borrar. Si un nodo responde not primary, uso el nodo lider.

docker exec -it mongo3 mongosh --eval

"db.getSiblingDB('banco').cuentas.deleteMany({})"

Borra los documentos desde mongo3 si mongo3 es PRIMARY.

Tengo alternativas porque el PRIMARY puede cambiar despues de una recuperacion.

docker exec -it cassandra1 cqlsh -e "TRUNCATE redsocial.posts;"

Elimina todos los registros de la tabla posts.

TRUNCATE limpia la tabla de la red social para repetir la prueba.

docker compose down

Apaga y elimina los contenedores y la red de Compose, conservando datos si hay volumenes.

Este comando termina la maqueta cuando ya acabe la demo.

docker compose down -v

Apaga la maqueta y borra tambien los volumenes de datos.

Este comando es para empezar desde cero, como si fuera una maquina limpia.

8. Errores comunes que puedes explicar

Mensaje	Que significa	Que respondo
no replset config has been received	MongoDB arranco, pero aun no se inicializo el Replica Set.	Debo ejecutar rs.initiate una sola vez en una maquina limpia.
already initialized	El Replica Set ya fue creado antes.	No es un error grave; continuo con rs.status.
not primary	Estoy intentando escribir en un SECONDARY.	MongoDB solo permite escrituras en el PRIMARY, por eso reviso rs.status.
container is not running	El contenedor esta apagado.	Reviso docker ps -a y logs para saber por que se detuvo.
Exited (137)	Docker mato el contenedor, normalmente por memoria.	Por eso en el yml Cassandra usa version 4.1 y heap limitado.

9. Conclusion corta para cerrar

MongoDB representa CP porque, ante una particion, prefiere bloquear escrituras si no puede confirmar mayoria. Esto es adecuado para una aplicacion bancaria, donde un saldo inconsistente seria grave.

Cassandra representa AP porque, usando CONSISTENCY ONE, sigue aceptando escrituras aunque un nodo este aislado. Esto encaja con una red social, donde es preferible permitir publicaciones y sincronizar despues.